

Phase 6 — Step-by-Step Implementation

1 — Preparation (org objects / fields)

1. Ensure objects and fields exist:
 - **Order__c** (Customer Order) with fields: Shipping_Latitude__c, Shipping_Longitude__c, Vehicle__c (lookup), Recommended_Dealer__c (Lookup to Account/Dealer).
 - **Dealer__c** (or Account record type) with fields: Latitude__c, Longitude__c, Stock__c (JSON or related Stock object), Is_Active__c.
 - **Stock__c** (optional) with Dealer__c, Vehicle__c, Quantity__c, Last_Updated__c.
 2. Give Apex class access to appropriate profiles (via security settings) and set CORS if necessary for external APIs.
-

2 — Lightning App Builder: create “Order Management Dashboard”

1. Setup → App Builder → **New** → Select **App Page** → name: **Order Management Dashboard**.
 2. Choose a layout (e.g., 3-column) and save.
 3. Add the following components to the page:
 - **Record Detail** or custom **Customer Order** component (center).
 - Custom LWC: **nearest-dealer-locator** (left / map).
 - Custom LWC: **stock-availability** (right).
 - Related Lists: Dealers, Vehicles, Bulk Orders.
 4. Activate the page and assign to app/profile.
-

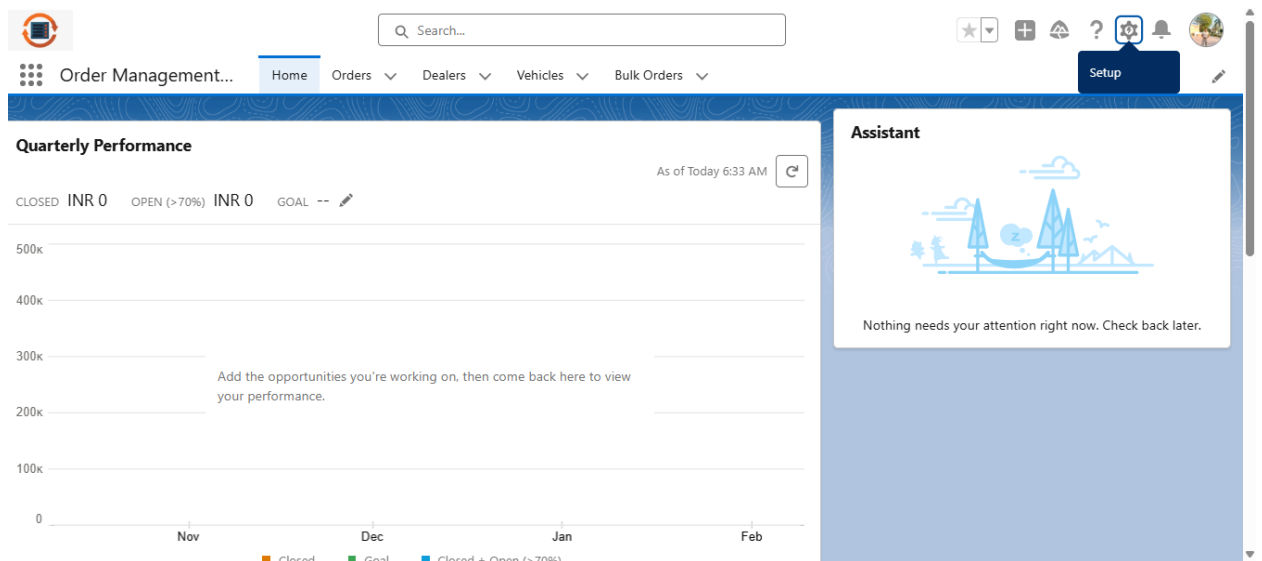
3 — Customer Order Record Page with stock availability

1. Setup → Object Manager → Order__c → Lightning Record Pages → New → Clone Lightning Record Page (or create).
2. Add sections:
 - Top: Order detail (compact layout).

- Side: **stock-availability** LWC — configure component properties to accept recordId.
3. Save & Activate for desired app and profiles.

4 — Tabs: Orders, Dealers, Vehicles, Bulk Orders

1. Setup → App Manager → Select your app (e.g., Order Management) → Edit.
2. In app navigation, add tabs:
 - Orders (Order__c list view)
 - Dealers (Dealer__c list view)
 - Vehicles (Vehicle__c)
 - Bulk Orders (custom list or VF/LWC page for bulk upload)
3. Save.



5 — Utility Bar: Quick dealer lookup tool

1. Setup → App Manager → Edit App → Utility Bar → Add Utility Item:
 - Type: **Custom Lightning Component** → point to dealer-quick-lookup LWC.
 - Configure icon, height.

2. The utility LWC will allow searching a dealer by name or quick nearby search using current order coordinates. (example LWC included below)

6 — Apex controller (real-time dealer + stock info)

Create Apex class DealerController.cls:

```
public with sharing class DealerController {

    @AuraEnabled(cacheable=true)

    public static List<DealerDTO> getNearestDealers(Decimal lat, Decimal lon, Integer limitSize) {

        if (lat == null || lon == null) return new List<DealerDTO>();

        // Haversine or use geolocation fields if present. Example simplified SOQL using geolocation field
        'Location__c'

        // Replace with correct field names

        Integer lim = limitSize == null ? 5 : limitSize;

        List<Dealer__c> dealers = [

            SELECT Id, Name, Latitude__c, Longitude__c, Is_Active__c

            FROM Dealer__c

            WHERE Is_Active__c = true

            LIMIT :lim

        ];

        List<DealerDTO> result = new List<DealerDTO>();

        for (Dealer__c d : dealers) {

            DealerDTO dto = new DealerDTO();

            dto.id = d.Id;

            dto.name = d.Name;

            dto.latitude = (Double) d.Latitude__c;

            dto.longitude = (Double) d.Longitude__c;
```

```

        // optionally compute distance here or let client compute
        result.add(dto);
    }
    return result;
}

```

@AuraEnabled

```

public static StockDTO getStockForDealerAndVehicle(Id dealerId, Id vehicleId) {
    StockDTO dto = new StockDTO();
    if (dealerId == null || vehicleId == null) return dto;
    Stock__c st = [
        SELECT Dealer__c, Vehicle__c, Quantity__c, Last_Updated__c
        FROM Stock__c
        WHERE Dealer__c = :dealerId AND Vehicle__c = :vehicleId
        LIMIT 1
    ];
    dto.dealerId = st.Dealer__c;
    dto.vehicleId = st.Vehicle__c;
    dto.quantity = (Integer) st.Quantity__c;
    dto.lastUpdated = st.Last_Updated__c;
    return dto;
}

```

// DTOs

```

public class DealerDTO {

```

```

    @AuraEnabled public Id id;

    @AuraEnabled public String name;

    @AuraEnabled public Double latitude;

    @AuraEnabled public Double longitude;
}

```

```

public class StockDTO {

    @AuraEnabled public Id dealerId;

    @AuraEnabled public Id vehicleId;

    @AuraEnabled public Integer quantity;

    @AuraEnabled public Datetime lastUpdated;

}

}

```

Notes: replace field names with your actual schema; use geolocation fields or an external distance calculation if you have many records (consider SOSL/third-party geo index or batch).

7 — LWC #1: nearest-dealer-locator (map + select nearest)

Files to create: nearestDealerLocator.html, nearestDealerLocator.js, nearestDealerLocator.js-meta.xml

nearestDealerLocator.html

```

<template>

  <lightning-card title="Nearest Dealers">

    <div class="slds-p-around_small">

      <template if:true={hasLocation}>

        <lightning-input type="number" label="Search radius (km)" value={radius}
onchange={handleRadiusChange}></lightning-input>

        <lightning-button label="Find Dealers" onclick={findDealers} class="slds-m-top_small"></lightning-
button>

```

```

<template if:true={dealers}>

    <lightning-map map-markers={mapMarkers} center={mapCenter} zoom-level="10" class="slds-m-top_small"></lightning-map>

    <div class="slds-m-top_small">

        <template for:each={dealers} for:item="d">

            <div key={d.id} class="slds-box slds-m-bottom_x-small">

                <strong>{d.name}</strong>

                <div>Distance: {d.distance} km</div>

                <lightning-button label="Recommend" data-id={d.id} onclick={handleRecommendClick} variant="brand"></lightning-button>

            </div>

        </template>

    </div>

</template>

</div>

</template>

</template>

<template if:false={hasLocation}>

    <div>No order location available.</div>

</template>

</div>

</lightning-card>

</template>

```

nearestDealerLocator.js

```

import { LightningElement, api, track } from 'lwc';

import getNearestDealers from '@salesforce/apex/DealerController.getNearestDealers';

import { publish, MessageContext } from 'lightning/messageService';

import ORDER_CHANNEL from '@salesforce/messageChannel/OrderChannel__c';

```

```
import { NavigationMixin } from 'lightning/navigation';

export default class NearestDealerLocator extends NavigationMixin(LightningElement) {

    @api recordId; // Order Id

    @api latitude;

    @api longitude;

    @track dealers = [];

    @track mapMarkers = [];

    @track mapCenter;

    @track radius = 50;

    @track hasLocation = false;

    @wire(MessageContext) messageContext;

    connectedCallback() {

        // If record supplies coords as attributes

        if (this.latitude && this.longitude) {

            this.hasLocation = true;

            this.mapCenter = { location: { Latitude: this.latitude, Longitude: this.longitude }};

        } else {

            this.hasLocation = false;

        }

    }

    handleRadiusChange(event) {
```

```
this.radius = event.target.value;
}

async findDealers() {
  try {
    const dealers = await getNearestDealers({ lat: this.latitude, lon: this.longitude, limitSize: 10 });
    // compute simple distance (client side) and prepare map markers
    const markers = [];
    const list = dealers.map(d => {
      const dist = this.computeDistance(this.latitude, this.longitude, d.latitude, d.longitude);
      markers.push({
        location: { Latitude: d.latitude, Longitude: d.longitude },
        title: d.name,
        description: `${dist.toFixed(1)} km`,
        id: d.id
      });
      return { ...d, distance: dist.toFixed(1) };
    }).sort((a,b)=> a.distance - b.distance);

    this.dealers = list;
    this.mapMarkers = markers;
  } catch (err) {
    // handle errors
    console.error(err);
  }
}
```



```
}
```

```
computeDistance(lat1, lon1, lat2, lon2){  
  // Haversine formula  
  const R = 6371; // km  
  const toRad = x => x * Math.PI / 180;  
  const dLat = toRad(lat2 - lat1);  
  const dLon = toRad(lon2 - lon1);  
  const a = Math.sin(dLat/2) * Math.sin(dLat/2) +  
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *  
    Math.sin(dLon/2) * Math.sin(dLon/2);  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));  
  return R * c;  
}
```

```
handleRecommendClick(event) {  
  const dealerId = event.target.dataset.id;  
  
  // Publish Lightning Message Service event to other components  
  const payload = { orderId: this.recordId, recommendedDealerId: dealerId };  
  publish(this.messageContext, ORDER_CHANNEL, payload);  
  
  // Navigate to dealer record  
  this[NavigationMixin.Navigate]({  
    type: 'standard__recordPage',  
    attributes: {
```

```
    recordId: dealerId,  
    objectApiName: 'Dealer__c',  
    actionName: 'view'  
  }  
});  
}  
}
```

nearestDealerLocator.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">  
  <apiVersion>58.0</apiVersion>  
  <isExposed>true</isExposed>  
  <masterLabel>Nearest Dealer Locator</masterLabel>  
  <description>Shows nearest dealers on a map and recommends a dealer</description>  
  <targets>  
    <target>lightning__RecordPage</target>  
    <target>lightning__AppPage</target>  
    <target>lightning__HomePage</target>  
  </targets>  
  <targetConfigs>  
    <targetConfig targets="lightning__RecordPage">  
      <property name="recordId" type="String" />  
      <property name="latitude" type="Number" label="Order Latitude" />  
      <property name="longitude" type="Number" label="Order Longitude" />  
    </targetConfig>
```

```
</targetConfigs>

</LightningComponentBundle>
```

8 — LWC #2: stock-availability (indicator + live updates)

Purpose: show stock for the Order__c.Vehicle__c at selected dealer and update in real time via LMS.

stockAvailability.html

```
<template>

  <lightning-card title="Stock Availability">

    <div class="slds-p-around_small">

      <template if:true={stockInfo}>

        <div>Dealer: {stockInfo.dealerName}</div>

        <div>Vehicle: {stockInfo.vehicleName}</div>

        <div>Status: <strong>{statusLabel}</strong></div>

        <div>Quantity: {stockInfo.quantity}</div>

        <div>Last updated: {stockInfo.lastUpdated}</div>

      </template>

      <template if:false={stockInfo}>

        <div>No stock data available.</div>

      </template>

    </div>

  </lightning-card>

</template>
```

stockAvailability.js

```
import { LightningElement, api, track, wire } from 'lwc';

import getStockForDealerAndVehicle from
  '@salesforce/apex/DealerController.getStockForDealerAndVehicle';
```

```

import { subscribe, MessageContext } from 'lightning/messageService';

import ORDER_CHANNEL from '@salesforce/messageChannel/OrderChannel__c';

export default class StockAvailability extends LightningElement {

  @api recordId; // order id

  @track stockInfo;

  subscription = null;

  @wire(MessageContext) messageContext;

  connectedCallback() {

    // subscribe to LMS order channel to receive recommended dealer

    this.subscription = subscribe(this.messageContext, ORDER_CHANNEL, (message) => {

      if (message && message.recommendedDealerId && message.orderId === this.recordId) {

        this.fetchStock(message.recommendedDealerId);

      }

    });

    // Optionally, initial fetch based on Recommended_Dealer__c field on order

    // (You can wire a getRecord for Order and call fetchStock if present)

  }

  async fetchStock(dealerId) {

    // Get vehicle id from order — simplified: assume parent sets record-level vehicle via attribute

    // if you need to get vehicle id, wire getRecord on Order__c

    const order = this.recordId;

```

```

// For demo, we assume event includes vehicleId; otherwise fetch order record first
try {
  // Example: dealerId known, vehicle id retrieved via other mechanism
  const vehicleId = this.vehicleId || null;

  if (!vehicleId) {
    // get vehicle id via getRecord or include in message from locator
    return;
  }

  const res = await getStockForDealerAndVehicle({ dealerId: dealerId, vehicleId: vehicleId });
  if (res) {
    this.stockInfo = {
      dealerId: res.dealerId,
      vehicleId: res.vehicleId,
      quantity: res.quantity,
      lastUpdated: res.lastUpdated,
      dealerName: res.dealerName || 'Selected Dealer',
      vehicleName: res.vehicleName || 'Selected Vehicle'
    };
  }
} catch (err) {
  console.error(err);
}

}

getStatusLabel() {

```

```
    if (!this.stockInfo) return 'Unknown';

    if (this.stockInfo.quantity > 5) return 'In Stock';

    if (this.stockInfo.quantity > 0) return 'Low Stock';

    return 'Out of Stock';

}

}
```

stockAvailability.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

    <apiVersion>58.0</apiVersion>

    <isExposed>true</isExposed>

    <masterLabel>Stock Availability</masterLabel>

    <targets>

        <target>lightning__RecordPage</target>

        <target>lightning__AppPage</target>

    </targets>

    <targetConfigs>

        <targetConfig targets="lightning__RecordPage">

            <property name="recordId" type="String" />

        </targetConfig>

    </targetConfigs>

</LightningComponentBundle>
```

Important: stock LWC expects Vehicle Id. You can supply it via @api vehicleId (set as component property in app builder) or read Order__c with getRecord. For simplicity wire getRecord inside the component to pull Order__c.Vehicle__c on init.

9 — Cross-component events: Lightning Message Service (LMS)

1. Create a message channel OrderChannel.messageChannel-meta.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<MessageChannel xmlns="http://soap.sforce.com/2006/04/metadata">
```

```
  <masterLabel>Order Channel</masterLabel>
```

```
  <isExposed>true</isExposed>
```

```
  <description>Channel for order to dealer messages</description>
```

```
</MessageChannel>
```

2. Use publish (in locator) and subscribe (in stock LWC) as shown above.
3. Payload example: { orderId: 'a01...', recommendedDealerId: '001...', vehicleId: 'a0B...' }.